

CKA: Design and Install a Kubernetes Cluster

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Choosing Kubernetes Infrastructure

This guide helps you select the right Kubernetes infrastructure by exploring various hosting options and deployment strategies for local and production environments.

Welcome to this informative guide on selecting the right Kubernetes infrastructure for your needs. In this article, we explore various hosting options for Kubernetes clusters, examine their unique characteristics, and provide insights into both local and production deployments. Whether you're new to Kubernetes or looking to expand your production environment, this guide will help you make an informed decision.

Kubernetes can be deployed on numerous platforms—from your local machine or laptop to both physical and virtual servers hosted on-premises or in the cloud. Your choice will depend on your technical requirements, cloud ecosystem compatibility, and the type of applications you plan to run.

Deploying Kubernetes on a Local Machine

For local development or testing purposes, several setup strategies are available. On a supported Linux system, you can install the Kubernetes binaries manually to configure a local cluster. However, for beginners, this process may be challenging, and a more automated solution is generally preferred to simplify cluster setup.

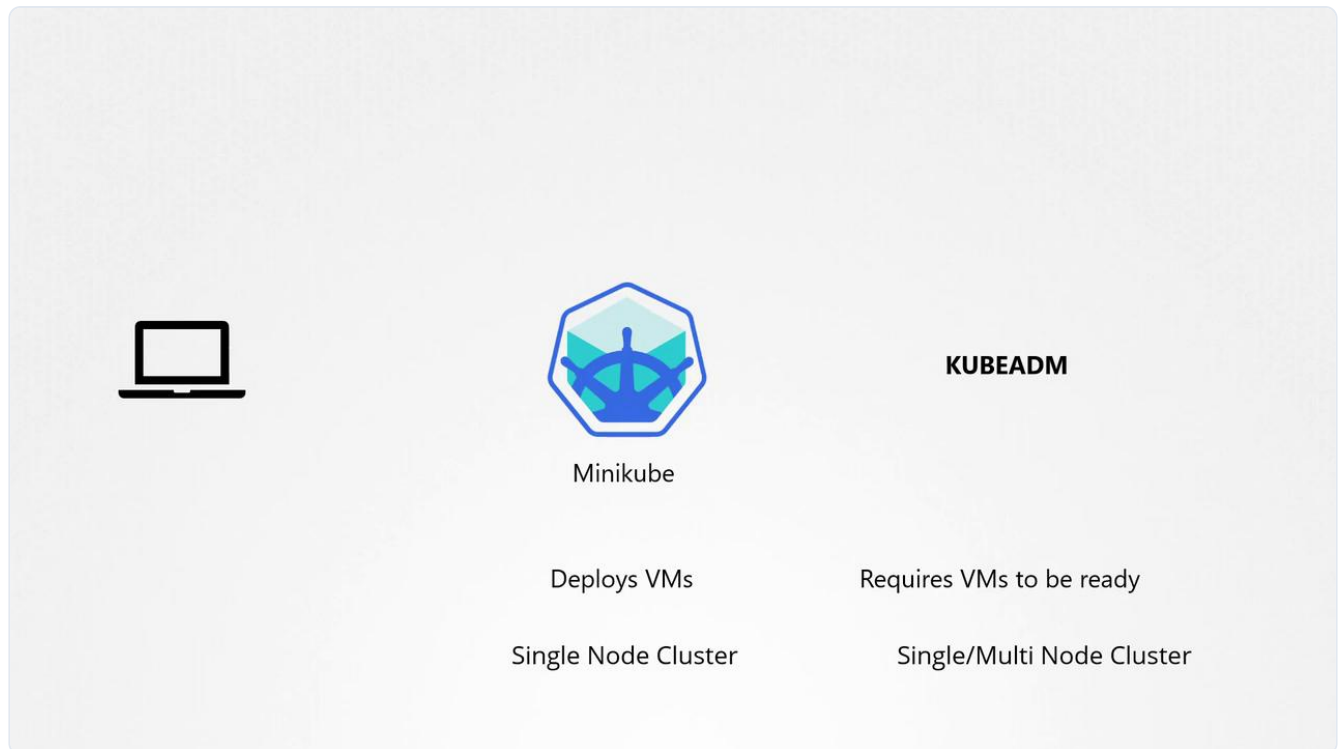
On Windows, since native Kubernetes binaries are not available, you need to rely on virtualization platforms such as Hyper-V, VMware Workstation, or VirtualBox. Windows users typically create Linux virtual machines (VMs) to host Kubernetes. Although Kubernetes components can run as Docker containers on these VMs, they operate on minimal Linux operating systems provided by Hyper-V.

Local Deployment Solutions

Two popular local deployment methods include:

- **Minikube:** This tool deploys a single-node Kubernetes cluster effortlessly by creating and managing VMs with virtualization software like Oracle VirtualBox. Minikube automatically provisions the VM with the necessary configuration.
- **Kubeadm:** This tool can quickly bootstrap both single-node and multi-node clusters but requires that you have already provisioned the VMs beforehand.

Local Kubernetes deployments on laptops are ideal for learning, development, and testing scenarios.



Production Deployment Options

In production environments, Kubernetes clusters are typically deployed in private or public clouds. Production solutions fall into two main categories: turnkey solutions and hosted (or managed) solutions.

Turnkey Solutions

Turnkey solutions enable you to provision and configure Kubernetes clusters with automated tools or scripts. While these solutions automate the deployment process, you remain responsible for maintaining, patching, and upgrading the underlying VMs. For example, deploying a Kubernetes cluster on AWS using the KOPS tool automates much of the setup.

Turnkey solutions require careful management of the underlying infrastructure, so ensure you monitor and update your VMs regularly.

Hosted Solutions

Hosted solutions provide Kubernetes as a service, where the provider manages the entire cluster infrastructure, including VM provisioning, maintenance, and configuration. This approach greatly simplifies the deployment process. For instance, Google Container Engine (GKE) lets you deploy a Kubernetes cluster in minutes with minimal manual intervention.

Turnkey Solutions

- You Provision VMs
- You Configure VMs
- You Use Scripts to Deploy Cluster
- You Maintain VMs yourself
- Eg: Kubernetes on AWS using KOPS

Hosted Solutions

(Managed Solutions)

- Kubernetes-As-A-Service
- Provider provisions VMs
- Provider installs Kubernetes
- Provider maintains VMs
- Eg: Google Container Engine (GKE)

Turnkey Solutions: On-Premises Options

For on-premises deployments, several turnkey solutions are available:

- **OpenShift:** Powered by Red Hat, OpenShift is a robust Kubernetes platform that comes with additional management tools and a graphical user interface for resource management and CI/CD integration. Check out the [OpenShift for Beginners](#) course for an in-depth introduction.
- **Cloud Foundry Container Runtime:** An open-source project that uses the BOSH tool to deploy and manage highly available Kubernetes clusters.
- **VMware Cloud PKS:** An excellent option for leveraging an existing VMware environment to deploy Kubernetes.
- **Vagrant:** Provides scripts to deploy Kubernetes clusters across various cloud service providers.

Turnkey Solutions



OpenShift



Cloud Foundry
Container Runtime



VMware Cloud
PKS



Vagrant



These turnkey options simplify the setup and management of Kubernetes clusters within your organization, assuming that your virtual machines meet the required specifications. For additional certified solutions and best practices, refer to the [Kubernetes Documentation](#).

Hosted Solutions: Cloud Offerings

Several cloud providers offer managed Kubernetes services, making it easier to run containerized applications without the overhead of managing the underlying infrastructure. Top hosted offerings include:

- **Google Container Engine (GKE)** on Google Cloud Platform.
- **OpenShift Online** from Red Hat, delivering a fully functional Kubernetes cluster.
- **Azure Kubernetes Service (AKS)** on Microsoft Azure.
- **Amazon Elastic Container Service for Kubernetes (EKS)** on AWS.

These services are designed to reduce operational complexity and support rapid scaling of applications.

Hosted Solutions



Google Container Engine **(GKE)**



OpenShift Online



Azure Kubernetes Service



Amazon Elastic Container Service for Kubernetes (EKS)

Our Deployment Choice

For learning and lab environments—especially for those without access to a public cloud account—we opted for a local deployment using VirtualBox. According to our recent poll, the majority of our users prefer VirtualBox for their lab setups.

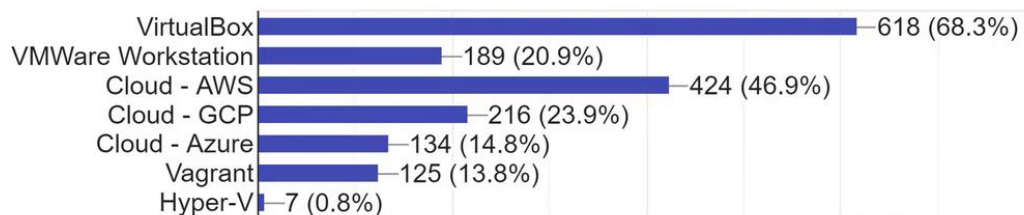
We deployed a local Kubernetes cluster from scratch by creating several VMs on VirtualBox. The design includes three nodes: one master and two worker nodes, all running on a single laptop.

Our Choice



Preferred Virtualization technology for Labs?

905 responses



When setting up your own lab using VirtualBox, ensure that your laptop meets the resource requirements for running multiple VMs efficiently.

This concludes our guide on choosing the right Kubernetes infrastructure. For further details on deployment options and best practices, please refer to additional documentation and trusted sources in the Kubernetes community.

For more in-depth reading:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

ETCD in HA

This lesson covers deploying ETCD in high-availability configurations, including cluster setup, data consistency, and the Raft protocol for Kubernetes resilience.

Welcome to this lesson on deploying ETCD in a high-availability (HA) configuration. In this guide, we will review ETCD fundamentals, explain how to configure a distributed cluster, and detail how data consistency is maintained across nodes using the Raft protocol. This HA setup is a critical requirement for running Kubernetes in a resilient manner.

What is ETCD?

ETCD is a distributed, reliable key-value store that is both fast and secure. Unlike traditional databases that store data in tables, ETCD organizes data as documents or pages. Each document holds all necessary information about a specific entity and can be formatted in JSON, YAML, or other structures. Changing one document does not affect others, which makes ETCD an excellent choice for modern, scalable architectures.

Objectives

- What is ETCD?
- What is a Key-Value Store?
- How to get started quickly?
- How to operate ETCD?
- What is a distributed system?
- How ETCD Operates
- RAFT Protocol
- Best practices on number of nodes

Distributed ETCD Clusters

ETCD's design is inherently distributed. Picture three different servers each running an identical instance of ETCD. This redundancy ensures that if one server (or node) fails, the remaining nodes continue to have an accurate copy of the data.



ETCD supports both read and write operations on any node. However, write operations require careful coordination. While clients might send write requests to any node, only the leader node processes these writes. If a write request lands on a follower, it is forwarded to the leader. The write operation is only confirmed after the leader gets an acknowledgement from a majority of the nodes.

Consistent

READ/WRITE



READ/WRITE



READ/WRITE



Read operations are simpler since every node contains the same data. Despite this, write operations necessitate a leader election through the Raft consensus protocol.

READ

READ



READ



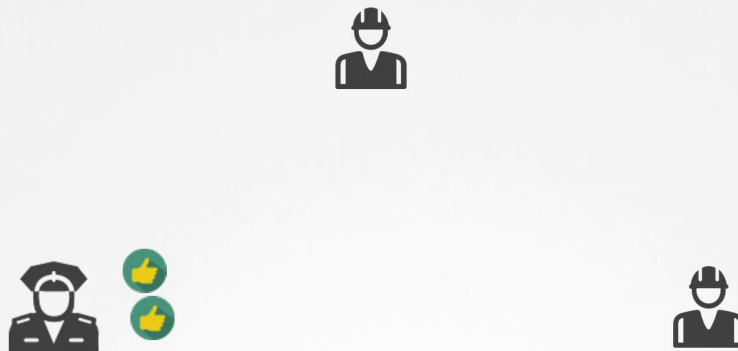
The Raft Consensus Protocol

Raft plays a crucial role in ensuring data consistency within an ETCD cluster. When the cluster boots up, no leader is present until one node's randomized timeout expires, triggering an election. During this election, the

candidate node requests votes from its peers. Once it obtains the necessary votes, it is crowned the leader and begins sending regular heartbeat messages to assert its control.

If the leader fails or experiences network issues, the remaining nodes automatically trigger a new election to establish a new leader. This robust process guarantees that all write requests are processed correctly and that every node's data remains synchronized.

Leader Election - RAFT



Quorum and Fault Tolerance

For a write operation to be successful in an ETCD cluster, it must be replicated to a majority of nodes, which is referred to as a quorum. For instance, in a three-node cluster, the update must reach at least two nodes. The quorum is calculated using the formula:

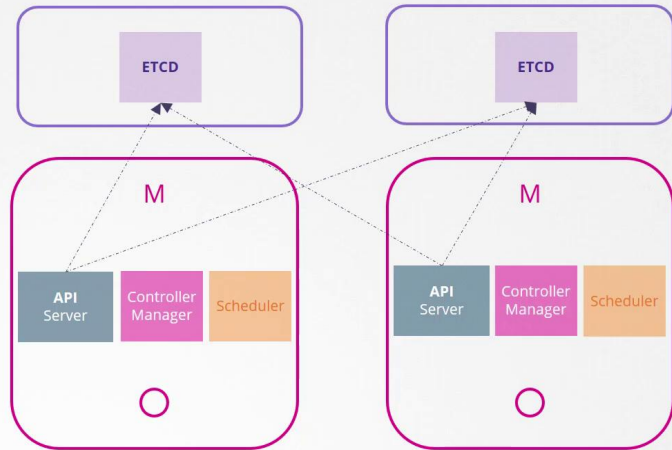
$$\text{quorum} = (\text{total number of nodes} / 2) + 1$$

This means a three-node cluster requires a quorum of 2, a five-node cluster requires 3, etc. Odd-numbered clusters are preferred because even-numbered ones may split into equal groups during network partitions, preventing either group from achieving quorum and potentially causing cluster failure.

Consider a six-node cluster: if a network partition results in subgroups of four and two, the larger group meets quorum and continues operation. However, if the split creates two groups of three, neither side meets the quorum of four nodes. In contrast, a seven-node cluster might split into groups of four and three, allowing the larger group to maintain functionality. For these reasons, odd node counts (e.g., three, five, seven) are strongly recommended for a robust HA cluster.

Number of Nodes

Instances	Quorum	Fault Tolerance
1	1	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3



Installing and Configuring ETCD

To install ETCD, follow these steps:

1. Download the latest supported binary.
2. Extract the downloaded archive.
3. Create the required directory structure.
4. Copy the generated certificate files to their designated locations.

For example, to download and extract the ETCD binary, use the following commands:

```
``bash theme={null} wget -q --https-only "https://github.com/coreos/etcd/releases/download/v3.3.9/etcd-v3.3.9-linux-amd64.tar.gz" tar -xvf etcd-v3.3.9-linux-amd64.tar.gz
```

Next, configure the ETCD service. A critical configuration parameter is the "initial cluster" option, which

```
```bash theme={null}
ExecStart=/usr/local/bin/etcd \
 --name ${ETCD_NAME} \
 --cert-file=/etc/etcd/kubernetes.pem \
 --key-file=/etc/etcd/kubernetes-key.pem \
 --peer-cert-file=/etc/etcd/kubernetes.pem \
 --peer-key-file=/etc/etcd/kubernetes-key.pem \
 --trusted-ca-file=/etc/etcd/ca.pem \
 --peer-trusted-ca-file=/etc/etcd/ca.pem \
 --peer-client-cert-auth \
 --client-cert-auth \
 --initial-advertise-peer-urls https://${INTERNAL_IP}:2380 \
 --listen-peer-urls https://${INTERNAL_IP}:2380 \
 --listen-client-urls https://${INTERNAL_IP}:2380,https://127.0.0.1:2379 \
 --advertise-client-urls https://${INTERNAL_IP}:2379 \
 --initial-cluster-token etcd-cluster-0 \
 --initial-cluster peer-1=https://${PEER1_IP}:2380,peer-2=https://${PEER2_IP}:2380 \
 --initial-cluster-state new \
 --data-dir=/var/lib/etcd
```

Ensure your certificate files and network configurations are correctly set up before starting the ETCD service.

## Using etcdctl

The command-line tool, etcdctl, is used to interact with the ETCD store by managing key-value pairs. This lesson uses the v3 API of etcdctl. Set the API version using the following command:

```
```bash theme={null} export ETCDCTL_API=3
```

You can add a key-value pair to the ETCD store with:

```
```bash theme={null}
etcdctl put name john
```

Then, retrieve the stored value using:

```
```bash theme={null} etcdctl get name
```

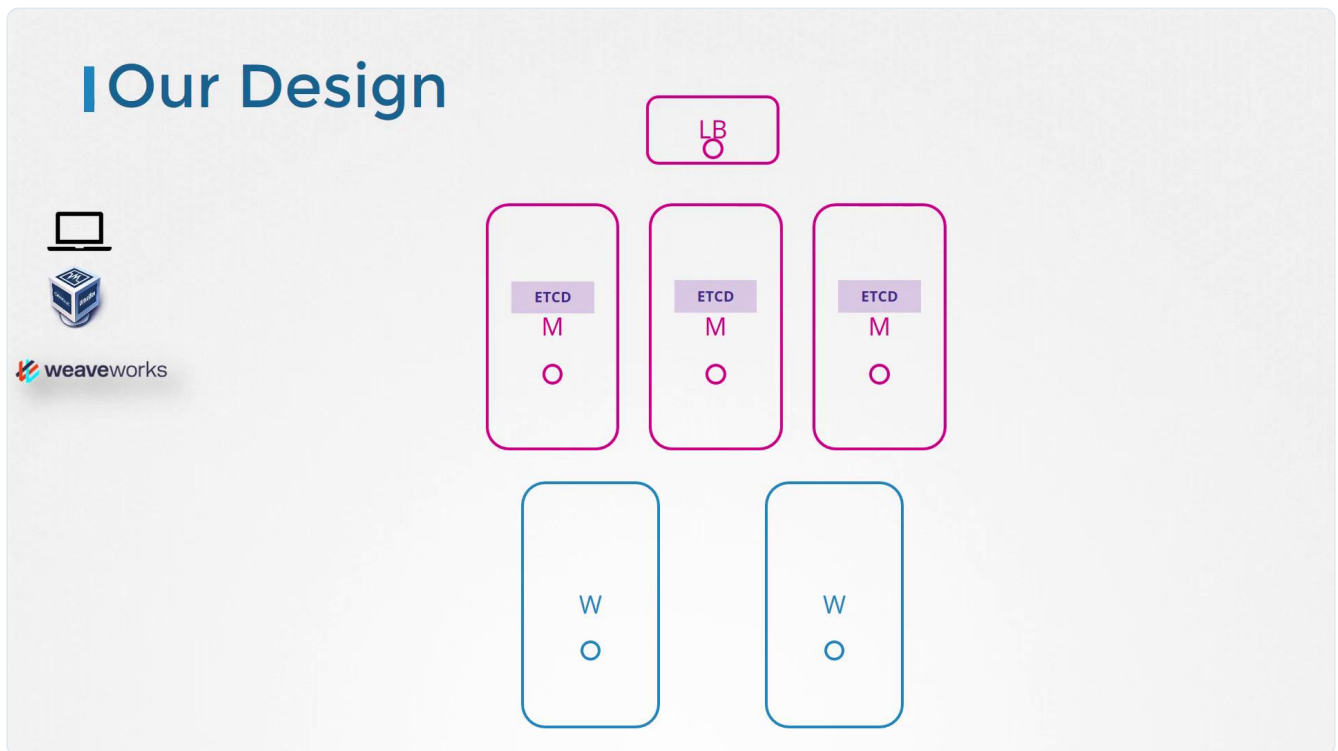
If successful, the output will be:

```
```text theme={null}
name
john
```

## Choosing the Right Cluster Size

For a highly available environment, a minimum cluster size of three nodes is required. Deploying only one or two nodes is insufficient as it compromises the quorum—if one node fails, the cluster will not have enough nodes to

operate. While a three-node cluster provides basic fault tolerance, a five-node cluster typically offers enhanced resilience without unnecessary complexity.



Deploying fewer than three nodes in production may lead to a split-brain scenario during network partitions, severely impacting cluster functionality.

For environments with limited resources, such as on a laptop, a two-node setup might be used for experimentation; however, this does not provide true fault tolerance. In production environments, always aim for a minimum of three nodes to maintain an effective HA configuration.

That concludes our lesson on deploying ETCD in a high-availability setup. Thank you for reading, and we look forward to seeing you in the next lesson.